

Sprawozdanie z zajęć laboratoryjnych: Algorytmy Ewolucyjne

Jan Taran

21 czerwca 2026

Spis treści

1	Wstęp	2
2	AE1 Podstawowy algorytm ewolucyjny	2
2.1	Opis zadania	2
2.2	Implementacja i operatory	2
2.3	Wyniki	3
3	AE2 – Wypełnianie koła prostokątami	4
3.1	Opis zadania	4
3.2	Kodowanie i operatory	4
3.3	Wyniki	4
4	AE3 Optymalizacja wag MLP algorytmem ewolucyjnym	5
4.1	Opis zadania	5
4.2	Kodowanie i operatory	5
4.3	Konfiguracja eksperymentów	6
4.4	Wyniki i porównanie z propagacją wsteczną	6
5	Wnioski	8

1 Wstęp

Celem niniejszego sprawozdania jest przedstawienie implementacji oraz analiza działania algorytmów ewolucyjnych (AE) zastosowanych do trzech zadań o rosnącym stopniu złożoności. W ramach zajęć zrealizowano:

- **AE1** podstawowy algorytm ewolucyjny do optymalizacji ciągłych funkcji wielowymiarowych (mutacja gaussowska, krzyżowanie jednopunktowe);
- **AE2** ewolucyjne rozwiązanie problemu pakowania (*cutting stock problem*) – wypełniania koła prostokątami o zadanych wartościach;
- **AE3** uczenie wag wielowarstwowego perceptronu (MLP) za pomocą algorytmu ewolucyjnego, wraz z porównaniem do klasycznej propagacji wstecznej błędu.

Wszystkie implementacje wykonano w języku Python, z wykorzystaniem biblioteki *NumPy* (jedynie do szybkich operacji macierzowych i funkcji losujących).

2 AE1 Podstawowy algorytm ewolucyjny

2.1 Opis zadania

Pierwsze zadanie polegało na implementacji podstawowego algorytmu ewolucyjnego minimalizującego nieznaną, wielowymiarową funkcję celu. Działanie algorytmu zweryfikowano na dwóch funkcjach testowych:

- prostej funkcji kwadratowej trójwymiarowej (jedno minimum globalne w punkcie $(0, 0, 0)$):

$$f(x, y, z) = x^2 + y^2 + 2z^2$$

- pięciowymiarowej funkcji Rastrigina – klasycznym benchmarku optymalizacji, charakteryzującym się ogromną liczbą minimów lokalnych przy jednym minimum globalnym w zerze:

$$f(\mathbf{x}) = 10n + \sum_{i=1}^n \left[x_i^2 - 10 \cos(2\pi x_i) \right], \quad n = 5$$

2.2 Implementacja i operatory

Algorytm zaimplementowano jako klasę `EvolutionaryAlgorithm` (plik `EA.py`). Osobnik reprezentowany jest jako wektor liczb rzeczywistych w przestrzeni $\mathbb{R}^d$, a populacja inicjalizowana jest losowo z rozkładu jednostajnego w zadanych granicach (`bounds`). Zastosowano następujące operatory:

- **Selekcja turniejowa** - z populacji losowanych jest `tournament_size` osobników, a do reprodukcji wybierany jest najlepszy z nich.
- **Krzyżowanie jednopunktowe** - z prawdopodobieństwem `crossover_rate` geny rodziców wymieniane są w losowo wybranym punkcie podziału (zaimplementowano również wariant uśredniający).

- **Mutacja gaussowska** - do każdego genu z prawdopodobieństwem `mutation_rate` dodawane jest zaburzenie z rozkładu $\mathcal{N}(0, \sigma)$, po czym osobnik jest przycinany do dozwolonych granic.
- **Elityzm** - najlepszy osobnik w każdym pokoleniu przechodzi bez zmian do następnej generacji.

```

1 def _gaussian_mutation(self, child):
2     for i in range(self.dimensions):
3         if np.random.rand() < self.mutation_rate:
4             child[i] += np.random.normal(0, self.sigma)
5     return np.clip(child, self.bounds[0], self.bounds[1])

```

Listing 1: Mutacja gaussowska z przycinaniem do granic (EA.py)

2.3 Wyniki

Dla populacji liczącej 100 osobników algorytm bez trudu zlokalizował minima obu funkcji, sprowadzając wartość funkcji celu praktycznie do zera (Tabela 1). Co istotne, algorytm poradził sobie również z silnie multimodalną funkcją Rastrigina, nie utykając w minimach lokalnych.

Tabela 1: Wyniki działania algorytmu AE1 na funkcjach testowych.

Funkcja	Wymiar	Znaleziona wartość minimum
$x^2 + y^2 + 2z^2$	3	$5.61 \cdot 10^{-8}$
Rastrigin	5	$9.65 \cdot 10^{-8}$

W celu doboru optymalnych hiperparametrów przeprowadzono przeszukiwanie siatki (*grid search*) obejmujące 60 kombinacji parametrów, każdą uruchomioną 5-krotnie (łącznie 300 przebiegów) na funkcji Rastrigina przy 1000 pokoleniach. Najlepsze wyniki zestawiono w Tabeli 2.

Tabela 2: Najlepsze konfiguracje hiperparametrów AE1 (funkcja Rastrigina, średnia z 5 przebiegów).

Crossover rate	Mutation rate	Sigma	Średnia wartość	Najlepsza wartość
0.75	0.20	0.5	$1.91 \cdot 10^{-10}$	$2.16 \cdot 10^{-11}$
0.50	0.20	0.5	$3.35 \cdot 10^{-10}$	$8.53 \cdot 10^{-11}$
0.75	0.20	1.0	$9.19 \cdot 10^{-10}$	—
1.00	0.20	0.5	$7.03 \cdot 10^{-9}$	—

Analiza wyników wskazuje, że kluczowy dla jakości rozwiązania okazał się stosunkowo wysoki współczynnik mutacji (0.20) oraz umiarkowana siła mutacji ($\sigma = 0.5$), co zapewniało właściwą równowagę między eksploracją przestrzeni a eksploatacją znalezionych rozwiązań. Zbadano także wpływ rozmiaru turnieju (`tournament_size`) na presję selekcyjną i tempo zbieżności algorytmu.

3 AE2 – Wypełnianie koła prostokątami

3.1 Opis zadania

Drugie zadanie stanowiło wariant problemu pakowania (*cutting stock problem*). Dane jest koło o promieniu r oraz zbiór typów prostokątów, z których każdy opisany jest trójką liczb: szerokością, wysokością i wartością. Celem jest takie ułożenie prostokątów wewnątrz koła, aby **zmaksymalizować** sumaryczną wartość, przy zachowaniu warunków: boki równoległe do osi, brak nakładania się wewnątrz prostokątów oraz możliwość wielokrotnego użycia każdego typu.

3.2 Kodowanie i operatory

Rozwiązanie zaimplementowano jako klasę `CuttingStockEA` (plik `RectanglesEA.py`).

- **Kodowanie:** chromosom to lista indeksów typów prostokątów (liczby całkowite). Określa ona *kolejność prób* wstawiania kolejnych prostokątów do koła. Maksymalną długość chromosomu oszacowano jako iloraz pola koła i pola najmniejszego prostokąta.
- **Dekodowanie (funkcja oceny):** chciwy algorytm umieszcza prostokąty wg kolejności z chromosomu, skanując dostępne pozycje. Aby przyspieszyć obliczenia, krok skanowania przyjęto jako największy wspólny dzielnik (NWD) wszystkich wymiarów prostokątów – jest on dostatecznie mały, by umożliwić idealne dopasowanie, a zarazem maksymalnie duży, by ograniczyć liczbę sprawdzeń.
- **Krzyżowanie ważone** (`_weighted_crossover`): dziecko dziedziczy geny od rodziców z prawdopodobieństwem proporcjonalnym do ich wartości funkcji celu (lepszy rodzic ma większy wpływ).
- **Mutacja:** każdy gen z prawdopodobieństwem `p_mutation` zostaje zastąpiony losowym indeksem typu prostokąta.
- **Selekcja i elityzm:** selekcja turniejowa (jeden z rodziców wybierany jest z większego turnieju, by faworyzować bardzo dobre osobniki); trzy najlepsze osobniki przechodzą bez zmian do następnego pokolenia.

3.3 Wyniki

Algorytm przetestowano na zestawach danych o różnych promieniach. We wszystkich przypadkach z określonym wymaganiem osiągnięto wynik znacząco przewyższający oczekiwany próg (Tabela 3).

Tabela 3: Wyniki AE2 – sumaryczna wartość prostokątów uzyskana dla poszczególnych zbiorów danych.

Zbiór	Dł. chromosomu	Liczba pokoleń	Wynik AE	Wynik oczekiwany
r800	167	200	31 860	$\geq 30\,000$
r1000	261	200	31 260	$\geq 17\,500$
r1100	211	200	40 860	$\geq 25\,000$
r1200	376	75	43 080	$\geq 30\,000$
r850	1891	50	565 420	(brak progu)

Wszystkie wymagane progi zostały spełnione z dużym zapasem. Zaobserwowano, że algorytm bardzo szybko poprawia rozwiązanie w pierwszych pokoleniach, po czym tempo wzrostu maleje – zgodnie z chciwym charakterem dekodowania i rosnącym wysyceniem przestrzeni koła.

4 AE3 Optymalizacja wag MLP algorytmem ewolucyjnym

4.1 Opis zadania

Celem ostatniego zadania było zastąpienie klasycznego algorytmu uczenia sieci neuronowej (propagacji wstecznej) algorytmem ewolucyjnym. Architektura sieci jest ustalona z góry, a optymalizowaną funkcją celu jest odwzorowanie wektora wag sieci na wartość błędu na zbiorze uczącym. Uzyskane wyniki porównano z uczeniem tej samej sieci metodą propagacji wstecznej.

W eksperymentach wykorzystano **własną implementację sieci MLP**. Funkcje aktywacji oraz funkcja straty zależały od typu zadania: **Softmax** z entropią krzyżową dla klasyfikacji oraz funkcja liniowa z błędem średniokwadratowym (MSE) dla regresji. Szczegółowe parametry poszczególnych sieci zestawiono w Tabeli 4.

4.2 Kodowanie i operatory

Algorytm ewolucyjny zaimplementowano jako klasę `EvolutionaryNNTrainer` (plik `NN_AE.py`).

- **Kodowanie:** chromosom to pojedynczy, spłaszczony wektor liczb rzeczywistych zawierający wszystkie wagi i biasy sieci (sklejone warstwa po warstwie). Funkcje pomocnicze `flatten_weights` oraz `set_weights_from_vector` odpowiadają za konwersję między wektorem a strukturą wag sieci.
- **Funkcja celu (minimalizacja):** błąd sieci na zbiorze uczącym – entropia krzyżowa (klasyfikacja) lub MSE (regresja).
- **Krzyżowanie BLX- α** – standardowy operator dla reprezentacji rzeczywistej; każdy gen dziecka losowany jest z przedziału rozpiętego między genami rodziców, poszerzonego o czynnik α (zaimplementowano też wariant jednopunktowy).
- **Mutacja gaussowska** z opcjonalnym wygaszaniem siły mutacji σ w kolejnych pokoleniach (`sigma_decay`).

- Selekcja turniejowa oraz elityzm.

```

1 lo = np.minimum(parent1, parent2)
2 hi = np.maximum(parent1, parent2)
3 d = hi - lo
4 low, high = lo - self.blx_alpha * d, hi + self.blx_alpha * d
5 child1 = self.rng.uniform(low, high)
6 child2 = self.rng.uniform(low, high)

```

Listing 2: Krzyżowanie BLX- α (NN_AE.py)

4.3 Konfiguracja eksperymentów

Eksperymenty przeprowadzono na trzech zbiorach: **iris** (klasyfikacja 3 klas), **multimodal-large** (regresja 1D) oraz **auto-mpg** (regresja – przewidywanie zużycia paliwa). Dane były standaryzowane (cechy oraz – dla regresji – również wartość docelowa). Parametry sieci i algorytmów zestawiono w Tabeli 4. Wspólne parametry AE: **tournament_size** = 3, **crossover_rate** = 0.8, **mutation_rate** = 0.1.

Tabela 4: Parametry sieci MLP oraz algorytmów uczących w zadaniu AE3.

	iris	multimodal-large	auto-mpg
Architektura	4-8-8-3	1-5-10-5-1	7-16-32-16-1
Aktywacja (ukryte)	ReLU	Sigmoid	ReLU
Aktywacja (wyjście)	Softmax	Linear	Linear
EA: liczebność populacji	80	100	80
EA: liczba pokoleń	200	500	300
EA: σ (decay)	0.20 (–)	0.15 (0.997)	0.15 (0.998)
EA: krzyżowanie	BLX- α	BLX- α	BLX- α
BP: optymalizator	Adam	Adam	Adam
BP: learning rate	0.05	0.01	0.01
BP: batch size	16	64	16
BP: liczba epok	400	300	500

Liczba generacji i pokoleń została dobrana odpowiednio, aby sieć trenowana ewolucyjnie miała szanse mieć wyniki zbliżone do sieci trenowanej propagacją wsteczną.

4.4 Wyniki i porównanie z propagacją wsteczną

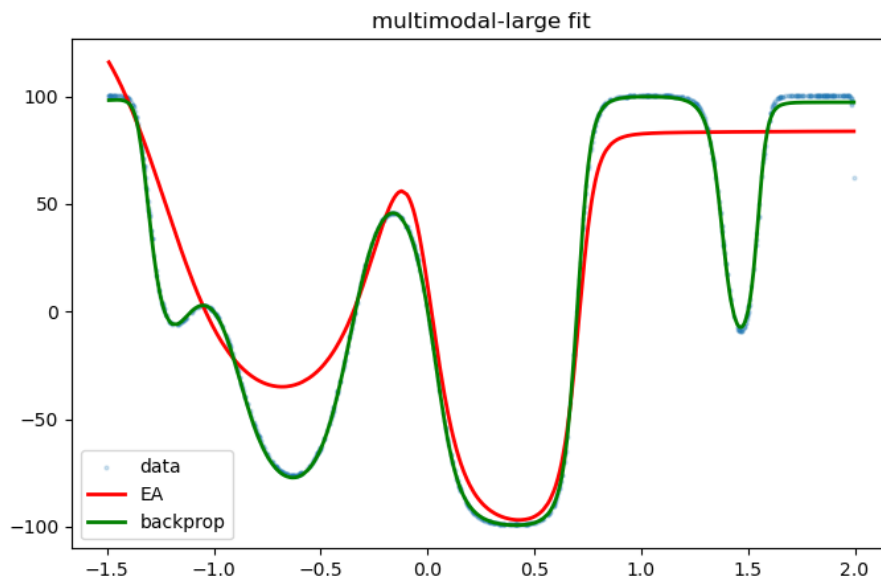
Zbiorcze porównanie obu metod przedstawiono w Tabeli 5. Jako punkt odniesienia podano błąd RMSE predyktora trywialnego (przewidyującego stałą średnią wartość).

Tabela 5: Porównanie wyników uczenia ewolucyjnego (EA) oraz propagacji wstecznej (BP) na zbiorze testowym.

Zbiór	Metryka	EA	BP	Czas EA [s]	Czas BP [s]
iris	Accuracy	0.978	0.978	2.9	0.6
multimodal-large	RMSE	39.883	1.705	228.6	13.7
auto-mpg	RMSE [mpg]	10.534	6.538	5.9	2.6

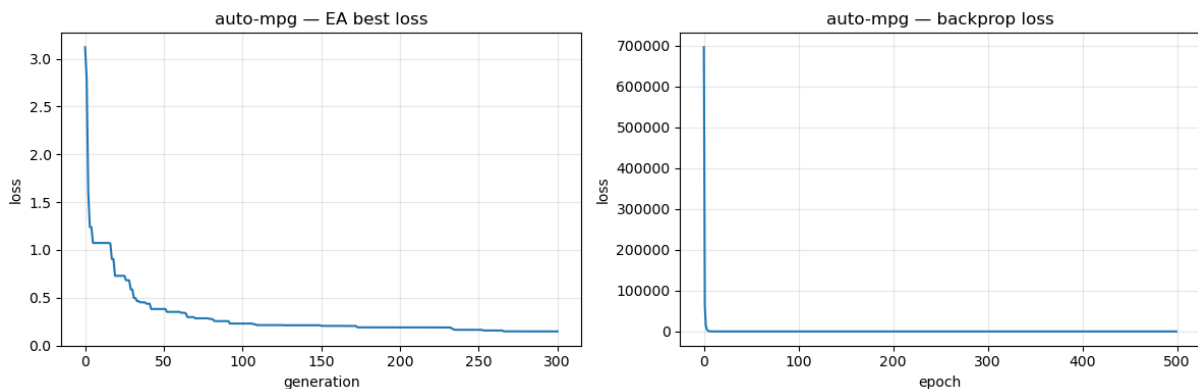
Klasyfikacja (iris). W zadaniu o niewielkiej liczbie wag algorytm ewolucyjny okazał się w pełni konkurencyjny – osiągnął identyczną trafność na zbiorze testowym jak propagacja wsteczna (0.978; tylko 1 błędna klasyfikacja na 45 obserwacji testowych), uzyskując nawet wyższą trafność na zbiorze treningowym (1.000 vs 0.981). Wynika to z niskiej wymiarowości przestrzeni przeszukiwania, w której operatory ewolucyjne działają skutecznie.

Regresja (multimodal-large). Na trudnym, silnie multimodalnym zbiorze regresyjnym ujawniła się przewaga uczenia gradientowego. Algorytm ewolucyjny istotnie zredukował błąd (RMSE 39.9), jednak nie zdołał dorównać precyzji propagacji wstecznej (RMSE 1.705). Jest to oczekiwane - w przypadku optymalizacji funkcji informacja o gradiencie jest przydatna.



Rysunek 1: Dopasowanie sieci (EA vs backprop) do zbioru multimodal-large

Regresja (auto-mpg). Na zbiorze o umiarkowanej liczbie wag różnica między metodami była niewielka (RMSE 10.5 dla EA wobec 6.5 dla BP).



Rysunek 2: Błąd w kolejnych epokach dla algorytmu ewolucyjnego i propagacji wstecznej

5 Wnioski

Przeprowadzone eksperymenty pozwalają sformułować następujące wnioski dotyczące zastosowania algorytmów ewolucyjnych:

Uniwersalność i prostota. Algorytm ewolucyjny okazał się niezwykle elastycznym narzędziem optymalizacji - ten sam, prosty schemat (selekcja, krzyżowanie, mutacja, elityzm) z powodzeniem zastosowano zarówno do ciągłej optymalizacji funkcji (AE1), problemu kombinatorycznego o reprezentacji całkowitoliczbowej (AE2), jak i do uczenia sieci neuronowej (AE3). Kluczem do sukcesu okazał się dobór reprezentacji osobnika oraz dopasowanych do niej operatorów genetycznych.

Skuteczność w problemach multimodalnych i kombinatorycznych. W zadaniach AE1 i AE2 algorytm wykazał dużą skuteczność - bezbłędnie zlokalizował minimum globalne silnie multimodalnej funkcji Rastrigina oraz znacząco przekroczył wszystkie wymagane progi wartości w problemie rozkroju. Potwierdza to przydatność AE tam, gdzie metody gradientowe są niestosowalne (przestrzeń dyskretna) lub zawodne (liczne minima lokalne).

Ograniczenia w optymalizacji wysokowymiarowej. Bezpośrednie porównanie z propagacją wsteczną (AE3) ujawniło naturalne ograniczenia podejścia ewolucyjnego. W przestrzeniach o niskiej wymiarowości (iris) AE dorównał metodzie gradientowej, jednak wraz ze wzrostem liczby wag jego skuteczność oraz efektywność czasowa wyraźnie ustępowały uczeniu wykorzystującemu informację o gradiencie. Algorytm ewolucyjny stanowi zatem wartościową alternatywę dla problemów nieróżniczkowalnych, lecz w klasycznym uczeniu sieci neuronowych propagacja wsteczna pozostaje metodą wydajniejszą.